

文本处理三剑客之 awk



awk 工作原理和基本用法说明

awk: Aho, Weinberger, Kernighan, 报告生成器, 格式化文本输出, GNU/Linux发布的AWK目前由自由软件基金会(FSF) 进行开发和维护, 通常也称它为 GNU AWK

awk有多种版本

- **AWK**: 原先来源于 AT & T 实验室的的AWK
- **NAWK**: New awk, AT & T 实验室的AWK的升级版
- **GAWK**: 即GNU AWK。所有的GNU/Linux发布版都自带GAWK, 它与AWK和NAWK完全兼容

目前主流发行版LINUX中使用的都是GAWK

```
# Rocky10
[root@localhost ~]# ll $(which awk)
lrwxrwxrwx. 1 root root 4 2024年10月29日 /usr/bin/awk -> gawk

# Ubuntu2404
magedu@Ubuntu2404:~$ which awk
/usr/bin/awk
magedu@Ubuntu2404:~$ ll /usr/bin/awk
lrwxrwxrwx 1 root root 21 Apr  9 2024 /usr/bin/awk -> /etc/alternatives/awk*
magedu@Ubuntu2404:~$ ll /etc/alternatives/awk*
lrwxrwxrwx 1 root root 13 Apr  9 2024 /etc/alternatives/awk -> /usr/bin/mawk*
lrwxrwxrwx 1 root root 29 Apr  9 2024 /etc/alternatives/awk.1.gz ->
/usr/share/man/man1/mawk.1.gz
```

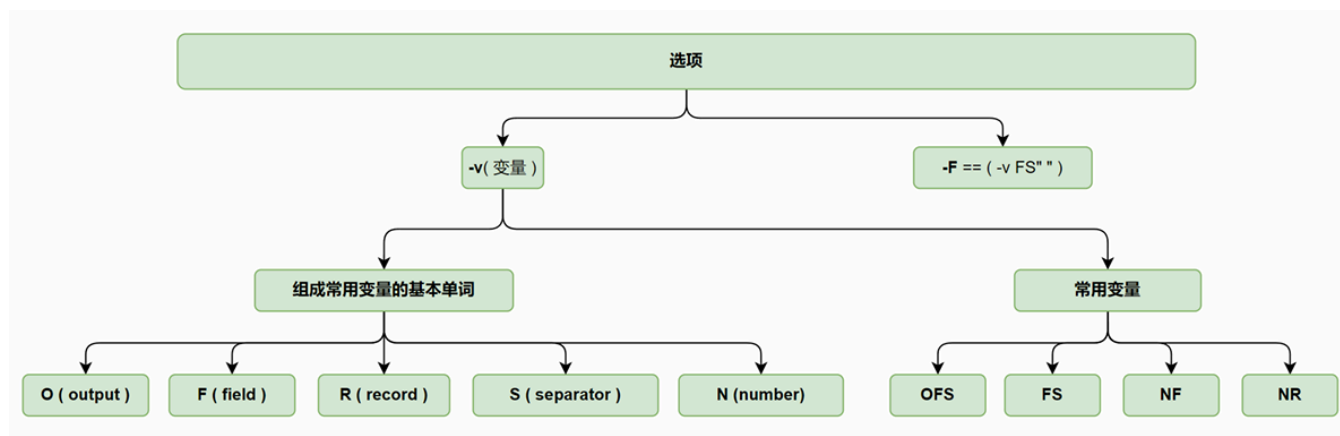
<https://www.gnu.org/software/gawk/manual/gawk.html>

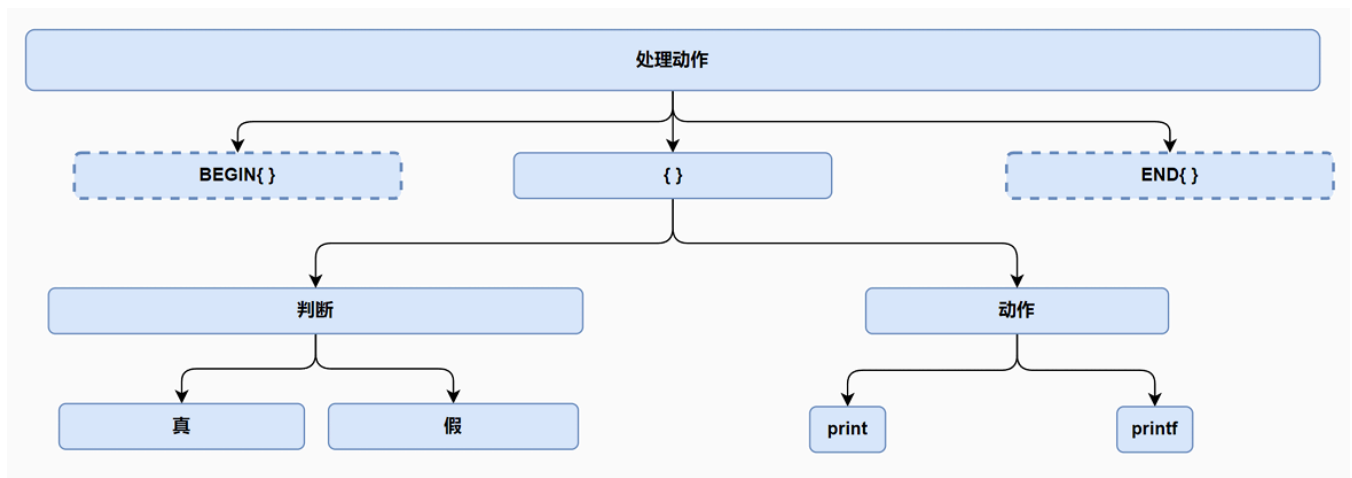
<https://man7.org/linux/man-pages/man1/awk.1p.html>

gawk：模式扫描和处理语言，可以实现下面功能

- 文本处理
- 输出格式化的文本报表
- 执行算术运算
- 执行字符串操作

awk基本语法





格式：

```
awk [options] var=value 'program' file
```

即 awk + 选项 + 处理动作 + file

处理动作 = 判断 + 动作

常用选项

-f progfile|--file progfile

-F fs|--field-separator fs

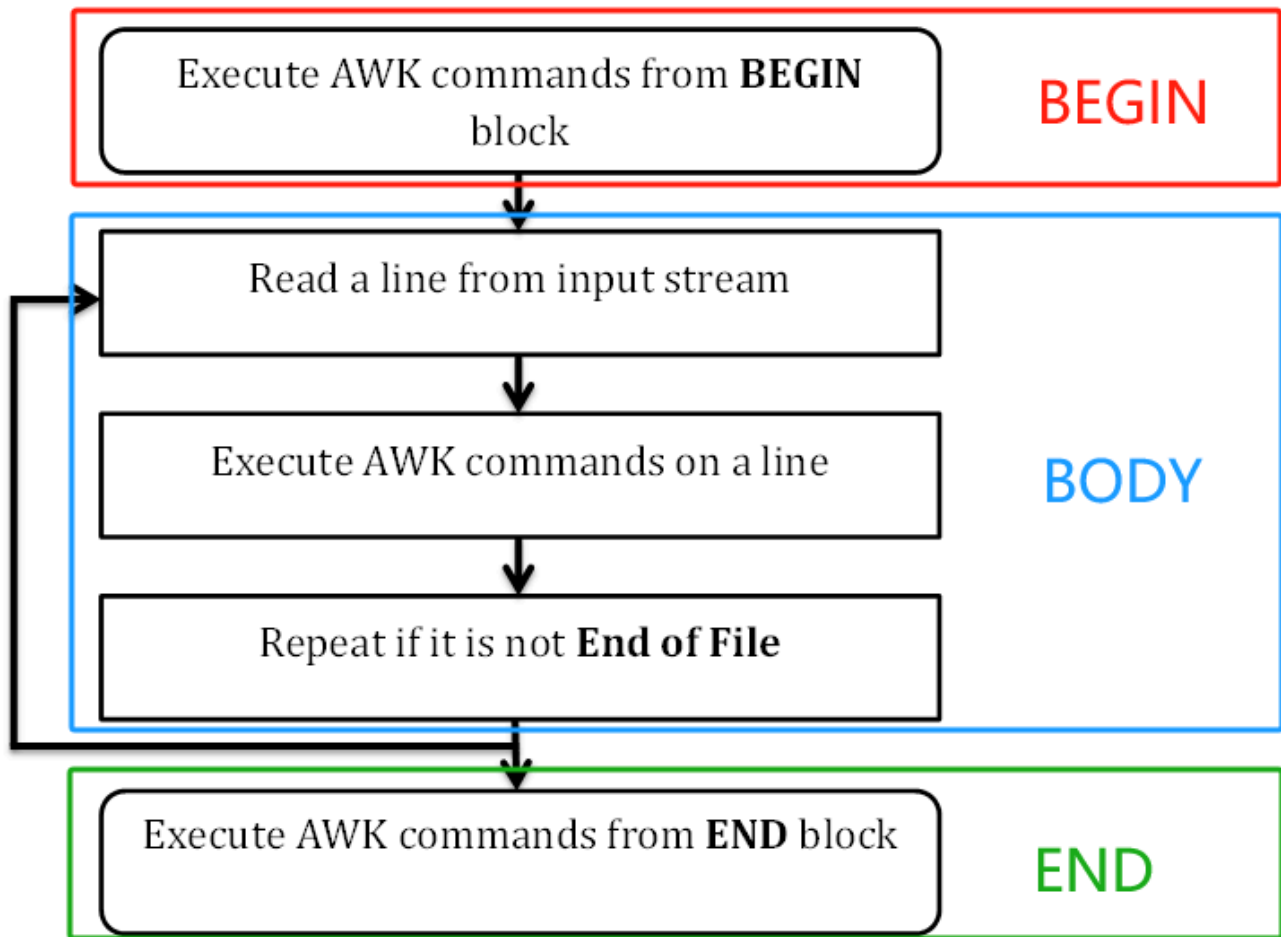
-v var=val|--assign var=val

从文件中读入Program

指定分隔符，默认是空白符，可以指定多个

设置变量

awk 工作过程



- 执行 `BEGIN{action;.....}` 语句块中的语句
- 从文件或标准输入(stdin)读取一行，然后执行 `pattern{action;.....}` 语句块，它逐行扫描文件，从第一行到最后一行重复这个过程，直到文件全部被读取完毕。
- 当读至输入流末尾时，执行 `END{action;.....}` 语句块

`BEGIN` 语句块在 awk 开始从输入流中读取行之前被执行，这是一个可选的语句块，比如变量初始化，打印输出表格的表头等语句通常可以写在 `BEGIN` 语句块中。

`END` 语句块在 awk 从输入流中读取完所有的行之后即被执行，比如打印所有行的分析结果这类信息总汇都是在 `END` 语句块中完成，它也是一个可选语句块。

`pattern` 语句块中的通用命令是最重要的部分，也是可选的。如果没有提供 `pattern` 语句块，则默认执行 `{print}`，即打印每一行读取到的行，awk 读取的每一行都会执行该语句块。

基本用法示例：

```
root@magedu:~# df | awk '{print}'
Filesystem              1k-blocks    Used Available Use% Mounted on
tmpfs                   396104      1540    394564    1% /run
/dev/mapper/ubuntu--vg-ubuntu--lv 101590008 6560772 89822608    7% /
tmpfs                   1980504        0    1980504    0% /dev/shm
tmpfs                    5120         0     5120    0% /run/lock
/dev/sda2               1992552   101840   1769472    6% /boot
tmpfs                   396100        12    396088    1% /run/
```

awk 变量

awk中的变量分为内置和自定义变量两种

变量的作用在于可以一次定义多次调用

内置变量

常用内置变量

变量名	含义
FILENAME	当前文件名
FS	字段分隔符，默认为空白字符，功能相当于 -F
RS	换行符，用于分割指定文件的行，默认是换行符
OFS	输出字段分隔符，默认为空白字符
ORS	输出换行符，输出时用指定符号代替换行符
OFMT	数字的输出格式，默认是 %.6g
NF	一条记录的字段数量
NR	已经读出的记录数，就是行号，从1开始
FNR	各文件分别记录行号
ARGC	命令行参数的个数
ARGV	数组，保存的是命令行所给定的各参数，每一个参数：ARGV[0]，...，ARGV[1]

范例：FILENAME

```
# begin中拿不到FILENAME
root@magedu:~# awk 'BEGIN{print FILENAME}' /etc/issue

root@magedu:~# awk 'BEGIN{print FILENAME}{print "test"}END{print FILENAME}' /etc/issue

test
test
/etc/issue
```

范例：FS

```
root@magedu:~# awk -v FS=":" 'BEGIN{print FS}{print $1,$2}' /etc/passwd
:
root x
daemon x

root@magedu:~# awk -v FS=":" 'BEGIN{print FS}{print $1FS$2}' /etc/passwd
:
root:x
daemon:x

# 使用-F选项指定
root@magedu:~# awk -F: 'BEGIN{print FS}{print $1FS$2}' /etc/passwd
:
root:x
daemon:x

# 从shell变量中获取
root@magedu:~# str=":";awk -v FS=$str 'BEGIN{print FS}{print $1FS$2}' /etc/passwd
:
root:x
daemon:x

# -F和FS变量功能一样，同时使用，后面会覆盖前面
root@magedu:~# awk -v FS=":" -F=" " 'BEGIN{print FS}' /etc/passwd
=
root@magedu:~# awk -v FS="=" -F":" 'BEGIN{print FS}' /etc/passwd
:
```

范例：同时使用多个分隔符

```
root@magedu:~# df -h | awk -F" +|%" '{print $5}'
Use
1
7
0
0
6
1
```

范例：RS

```
root@magedu:~# cat -A test.txt
a b c;1 2 3;x y z$

root@magedu:~# awk '{print $0}' test.txt
a b c;1 2 3;x y z

root@magedu:~# awk -v RS=";" '{print $0}' test.txt
a b c
1 2 3
x y z
```

范例：OFS

```
root@magedu:~# awk -v FS=":" '{print $1,$3}' /etc/passwd
root 0
daemon 1
bin 2

root@magedu:~# awk -v FS=":" -v OFS="---" '{print $1,$3}' /etc/passwd
root---0
daemon---1
bin---2
```

范例：ORS

```

root@magedu:~# cat -A test.txt
123$
456$
789$

root@magedu:~# awk '{print $0}' test.txt
123
456
789

root@magedu:~# awk -v ORS="---" '{print $0}' test.txt
123---456---789---root@magedu:~#

```

范例：OFMT

```

root@magedu:~# awk 'BEGIN{PI=3.1415926;print PI;OFMT="%.1g";print PI;OFMT="%.2g";print
PI;OFMT="%.8g";print PI;OFMT="%.8f";print PI}'
3.14159
3
3.1
3.1415926
3.14159260

```

范例：NF

```

root@magedu:~# awk -v FS=":" '{print NF}' /etc/passwd
7
7

root@magedu:~# awk -v FS=":" '{print $NF}' /etc/passwd
/usr/bin/bash
/usr/sbin/nologin

root@magedu:~# awk -v FS=":" '{print $(NF-1)}' /etc/passwd
/root
/usr/sbin

```

范例：NR

```

root@magedu:~# awk '{print NR, $0}' /etc/issue
1 Ubuntu 24.04.1 LTS \n \l
2

```

范例：FNR


```

root@magedu:~# awk '{print NR,FNR,$0}' /etc/issue /etc/os-release
1 1 Ubuntu 24.04.1 LTS \n \l
2 2
3 1 PRETTY_NAME="Ubuntu 24.04.1 LTS"
4 2 NAME="Ubuntu"
5 3 VERSION_ID="24.04"
6 4 VERSION="24.04.1 LTS (Noble Numbat)"
7 5 VERSION_CODENAME=noble
8 6 ID=ubuntu
9 7 ID_LIKE=debian
10 8 HOME_URL="https://www.ubuntu.com/"
11 9 SUPPORT_URL="https://help.ubuntu.com/"
12 10 BUG_REPORT_URL="https://bugs.launchpad.net/ubuntu/"
13 11 PRIVACY_POLICY_URL="https://www.ubuntu.com/legal/terms-and-policies/privacy-policy"
14 12 UBUNTU_CODENAME=noble
15 13 LOGO=ubuntu-logo

```

范例：ARGC

```

root@magedu:~# awk 'BEGIN{print ARGC}'
1
root@magedu:~# awk 'BEGIN{print ARGC}' /etc/issue
2
root@magedu:~# awk 'BEGIN{print ARGC}' /etc/issue /etc/os-release
3

```

范例：ARGV

```

root@magedu:~# awk 'BEGIN{print ARGC,"---",ARGV[0],"---",ARGV[1],"---",ARGV[2]}' /etc/issue
/etc/os-release
3 --- awk --- /etc/issue --- /etc/os-release

```

自定义变量

AWK中的自定义变量命名要区分大小写

用 `-v` 选项定义自定义变量

```
root@magedu:~# awk -v var1=abc 'BEGIN{print var1}'
abc

root@magedu:~# awk -v var1=abc -v VAR1=123 'BEGIN{print var1;print VAR1}'
abc
123
```

在 program 中定义

```
root@magedu:~# awk -v var1=abc 'BEGIN{print var1; var1=def; print var1}'
abc

root@magedu:~# awk -v var1=abc 'BEGIN{print var1;var1="def"; print var1}{print var1}'
/etc/issue
abc
def
def
def
```

动作 print

格式:

```
print item1, item2, ...
```

说明:

- 逗号分隔符
- 输出 `item` 可以是字符串, 也可以是数字, 当前记录的字段, 变量或awk的表达式
- 如省略item, 相当于print \$0
- 固定字符串需要用""引起来, 而变量和数字不需要

范例

```
root@magedu:~# awk 'BEGIN{print "hello world"}'

root@magedu:~# seq 3 |awk '{print "hello world"}'
hello world
hello world
hello world
```

```
root@magedu:~# seq 3 |awk '{print $0}'
1
2
3

root@magedu:~# awk 'BEGIN{print 2*3}'
6

root@magedu:~# awk -v var1=123 -F: '{print var1,$1,$3}' /etc/passwd
123 root 0
123 daemon 1
123 bin 2
123 sys 3
.....

# \t 是 制表符 (Tab)，其标准行为是：光标跳到下一个 8 的整数倍列位。
root@magedu:~# awk -v var1=123 -F: '{print var1,$1"\t"$3}' /etc/passwd
123 root    0
123 daemon  1
123 bin     2
123 sys     3
123 sync    4
```

动作 printf

printf 可以实现格式化输出

格式：

```
printf "FORMAT", item1, item2,...
```

说明：

- 必须指定FORMAT
- 不会自动换行，需要显式给出换行控制符\n
- FORMAT中需要分别为后面每个item指定格式符

格式符：与 `item` 一一对应

%s	# 显示字符串
%d %i	# 显示十进制整数
%f	# 显示为浮点数
%e %E	# 显示科学计数法数值
%c	# 显示字符的ASCII码
%g %G	# 以科学计数法或浮点形式显示数值
%u	# 无符号整数
%%	# 显示%自身

修饰符

M[.N]	# M表示显示的宽度，N表示小数点后精度，如%3.1f
-	# 左对齐（默认右对齐）如 %-15s
+	# 显示数值的正负号，如： %+d

范例：

```
root@magedu:~# awk -F: '{printf "%s", $1}' /etc/passwd
rootdaemonbinsyssyncgamesmanlpmailnewsuucpproxywww-databackuplistirc_aptnobodysystemd-
networksystemd-timesyncdhpcdmessagbussystemd-
resolvepollinatepolkitdsysloguidtdcpdumtsslandscapefwupd-
refreshusbmuxsshdmagedutompostfixjerrywangxiaohongulu2root@magedu:~#

root@magedu:~# awk -F: '{printf "%s\n", $1}' /etc/passwd
root
daemon
bin
sys
.....

root@magedu:~# awk -v var1=123 -F: '{printf "%s%20s\n", var1, $1$3}' /etc/passwd
123          root0
123          daemon1
123          bin2
.....

root@magedu:~# awk -F: '{printf "username: %-25sUID:%d\n", $1, $3}' /etc/passwd
username: root          UID:0
username: daemon       UID:1
username: bin          UID:2
username: sys          UID:3
```

范例：

```
root@magedu:~# awk -F: 'BEGIN{print "-----\n|username          |uid
|\n-----"}{printf "|%-20s|%5s|\n-----\n",$1,$3}'
/etc/passwd
-----
|username          |uid  |
-----
|root              |   0|
-----
|daemon            |   1|
-----
|bin                |   2|
-----
|sys                |   3|
-----
|sync              |   4|
-----
|games             |   5|
-----
|man                |   6|
-----
|lp                 |   7|
-----
|mail               |   8|
-----
.....
```

操作符

赋值操作

`=`, `+=`, `-=`, `*=`, `%=`, `^=`, `++`, `--`

比较操作

`==`, `!=`, `>`, `>=`, `<`, `<=`

算数运算符

`x+y`, `x-y`, `x*y`, `x/y`, `x^y`, `x%y`

`-x`

转换为负数

`+x`

将字符串转换为数值

范例：

```
# 默认print, 先匹配, 再++
root@magedu:~# seq 3|awk 'n++'
2
3

root@magedu:~# awk -v n=0 '!n++' /etc/passwd
root:x:0:0:root:/root:/usr/bin/bash

root@magedu:~# awk -v n=0 '!n++{print n}' /etc/passwd
1

root@magedu:~# awk -v n=1 '!n++{print n}' /etc/passwd

root@magedu:~# awk -v n=0 '!++n{print n}' /etc/passwd

root@magedu:~# awk -v n=0 '!++n' /etc/passwd

root@magedu:~# awk -v n=-1 '!++n' /etc/passwd
root:x:0:0:root:/root:/usr/bin/bash
```

范例:

```
root@magedu:~# awk 'NR==2' /etc/passwd
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin

root@magedu:~# awk -F: '$3>=1000' /etc/passwd
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
magedu:x:1000:1000:magedu:/home/magedu:/bin/bash
tom:x:1001:1001::/home/tom:/bin/sh
jerry:x:1002:1002::/home/jerry:/bin/sh
wang:x:1003:1003::/home/wang:/bin/sh
xiaohong:x:1004:1004::/home/xiaohong:/bin/sh
u1:x:1024:1024::/home/u1:/bin/bash
u2:x:1025:1025::/home/u2:/bin/bash
```

范例: 取奇, 偶数行

```
root@magedu:~# seq 6 |awk 'NR%2==0'
2
4
6

root@magedu:~# seq 6 |awk 'NR%2==1'
1
3
5

root@magedu:~# seq 6 |awk 'i!=i'
```

```
1
3
5

root@magedu:~# seq 6 | awk '!(i==i)'
```

2
4
6

模式匹配操作

```
~           # 左边是否和右边匹配，包含关系
!~         # 是否不匹配
```

范例

```
# 包含root的行
root@magedu:~# awk -F: '$0 ~ /root/{print $0}' /etc/passwd
root:x:0:0:root:/root:/usr/bin/bash

root@magedu:~# awk '/root/' /etc/passwd
root:x:0:0:root:/root:/usr/bin/bash

# 以root开头的行
root@magedu:~# awk -F: '$0 ~ /^root/{print $0}' /etc/passwd
root:x:0:0:root:/root:/usr/bin/bash

root@magedu:~# awk '/^root/' /etc/passwd
root:x:0:0:root:/root:/usr/bin/bash

# 不包含nologin的行
root@magedu:~# awk '$0 !~ /nologin/' /etc/passwd
root:x:0:0:root:/root:/usr/bin/bash
sync:x:4:65534:sync:/bin:/bin/sync
dhcpcd:x:100:65534:DHCP Client Daemon,,,:/usr/lib/dhcpcd:/bin/false
pollinate:x:102:1::/var/cache/pollinate:/bin/false
tss:x:106:108:TPM software stack,,,:/var/lib/tpm:/bin/false
magedu:x:1000:1000:magedu:/home/magedu:/bin/bash
tom:x:1001:1001::/home/tom:/bin/sh
jerry:x:1002:1002::/home/jerry:/bin/sh
wang:x:1003:1003::/home/wang:/bin/sh
xiaohong:x:1004:1004::/home/xiaohong:/bin/sh
u1:x:1024:1024::/home/u1:/bin/bash
u2:x:1025:1025::/home/u2:/bin/bash
```

范例：

```
root@magedu:~# df | awk -F"[:space:]]+|%" '$0 ~ /^\/dev\/sd/{print $5}'
6

root@magedu:~# ifconfig ens33 |awk 'NR==2{print $2}'
10.0.0.152
```

逻辑操作符

&&	# 与，并且关系
	# 或，或者关系
!	# 非，取反

范例：!取反

```
root@magedu:~# awk 'BEGIN{print i}'

root@magedu:~# awk 'BEGIN{print !i}'
1
root@magedu:~# awk -v i=10 'BEGIN{print !i}'
0
root@magedu:~# awk -v i=0 'BEGIN{print !i}'
1
root@magedu:~# awk -v i=abc 'BEGIN{print !i}'
0
root@magedu:~# awk -v i='' 'BEGIN{print !i}'
1
```

范例

```
root@magedu:~# awk -F: '$3>=0 && $3<5 {print $1,$3}' /etc/passwd
root 0
daemon 1
bin 2
sys 3
sync 4

root@magedu:~# awk -F: '$3<1 || $3>1010 {print $1,$3}' /etc/passwd
root 0
nobody 65534
u1 1024
u2 1025

root@magedu:~# awk -F: '!(($3!=0) {print $1,$3}' /etc/passwd
root 0
```



```
root@magedu:~# awk -F: '!(($3>=5) {print $1,$3}' /etc/passwd
root 0
daemon 1
bin 2
sys 3
sync 4
```

三目运算符（条件表达式）

```
selector?if-true-expression:if-false-expression
```

范例

```
root@magedu:~# awk -F: '{ $3 >= 1000 ? utype = "Common" : utype = "Sys"; printf
"%-20s:%12s\n", $1, utype }' /etc/passwd
daemon      :      Sys
bin         :      Sys
.....
nobody      :    Common
.....
```

范例：

```
[root@ubuntu2204 ~]# cat scores.txt
wang 100
li 90
zhang 50
zhao 80
han 70

[root@ubuntu2204 ~]# awk '$2 >= 60 ? type = "pass" : type = "nopass" {print $1, type}'
scores.txt
wang pass
li pass
zhang nopass
zhao pass
han pass
```

模式PATTERN

PATTERN：根据pattern条件，过滤匹配的行，再做处理，如果没有指定，则匹配每一行

范例：

```
root@magedu:~# awk '{print $0}' /etc/issue
Ubuntu 24.04.1 LTS \n \l

root@magedu:~# awk 'NR==1{print $0}' /etc/issue
Ubuntu 24.04.1 LTS \n \l
```

正则匹配

范例：用正则匹配，正则表达式需要用//来锚定开始结束

```
#非空行，非注释行
root@magedu:~# awk '!/^$|^#/{print $1}' /etc/fstab
/dev/disk/by-id/dm-uuid-LVM-f8Uc6ZZ0fgtnZU0fegs23fYybLJIVAovyINIBazS1TBdMHH8NcxRqfZcdcc2X1AS
/dev/disk/by-uuid/bd93c8be-9332-4d3d-921d-69dc9ba99002
/swap.img
```

关系表达式

关系表达式，结果为“真”才会被处理

真：结果为非0值，非空字符串

假：结果为0值或空字符串

范例：

```
root@magedu:~# seq 3 |awk '1'
1
2
3

root@magedu:~# seq 3 |awk '!1'
root@magedu:~# seq 3 |awk '0'

# false 这里表示的是一个未定义的变量，不是表示bool值
root@magedu:~# seq 3 |awk 'false'
root@magedu:~# seq 3 |awk -v false=123 'false'
1
2
```

```

3

# false这里表示的是字符串
root@magedu:~# seq 3 |awk '"false"'
1
2
3

# 空字符串为假，空格为真
root@magedu:~# seq 3 |awk '""'
root@magedu:~# seq 3 |awk '" "'
1
2
3

# 数字0为假，字符串0为真
root@magedu:~# seq 3 |awk '0'
root@magedu:~# seq 3 |awk '"0"'
1
2
3

# 这里true是变量名，一个不存在的变量
root@magedu:~# seq 3 |awk 'true'
root@magedu:~# seq 3 |awk -v true=123 'true'
1
2
3

```

BEGIN/END模式

BEGIN{ }: 仅在开始处理文件中的文本之前执行一次

END{ }: 仅在文本处理完成之后执行一次

范例：

```

#只有begin 时，可以没有输入内容
[root@ubuntu2204 ~]# awk 'BEGIN{print "begin"}'
begin

# END跟在{}后，所以必须有后续输入才行
[root@ubuntu2204 ~]#awk 'BEGIN{print "hello"}{'print'}END{print "world"}' /etc/issue
hello
Ubuntu 22.04.1 LTS \n \l

world

```

条件判断 if-else

使用场景：对awk取得的整行或某个字段做条件判断

语法

```
if(condition){statement;...}[else statement]
if(condition1){statement1}else if(condition2){statement2}else if(condition3)
{statement3}..... else {statementN}
```

范例：

```
[root@ubuntu2204 ~]#awk -F: '{if($3>=1000)print $1,$3}' /etc/passwd
nobody 65534
wang 1000

[root@ubuntu2204 ~]#awk -F: '{if($3<=100){print "<=100",$3}else if($3<=1000){print "
<=1000",$3} else{print ">=1000",$3}}' /etc/passwd |head -n20
<=100 0
<=100 1
<=100 2
<=100 3
<=100 4
<=100 5
<=100 6
<=100 7
<=100 8
<=100 9
<=100 10
<=100 13
<=100 33
<=100 34
<=100 38
<=100 39
<=100 41
>=1000 65534
<=100 100
<=1000 101
.....

[root@ubuntu2204 ~]#awk -F: '{if($NF==" /bin/bash"){print $1}}' /etc/passwd
root
wang
```

```
[root@ubuntu2204 ~]#awk -F: '{if($3>=1000) {printf "Common user: %s\n", $1} else {printf "root or Sysuser: %s\n", $1}}' /etc/passwd |head -n20
root or Sysuser: root
root or Sysuser: daemon
root or Sysuser: bin
root or Sysuser: sys
root or Sysuser: sync
root or Sysuser: games
root or Sysuser: man
root or Sysuser: lp
root or Sysuser: mail
root or Sysuser: news
root or Sysuser: uucp
root or Sysuser: proxy
root or Sysuser: www-data
root or Sysuser: backup
root or Sysuser: list
root or Sysuser: irc
root or Sysuser: gnats
Common user: nobody
root or Sysuser: _apt
root or Sysuser: systemd-network
```

```
[root@ubuntu2204 ~]#df -h |awk -F% '/^\/dev\/sd/{print $1}' |awk '$NF>=10{print $1,$5}'
/dev/sda2 14
```

```
[root@ubuntu2204 ~]#df |awk -F "[ %]+" '/^\/dev\/sd/{if($NF>10){print $(NF-1)" full"}
else{print $(NF-1)" ok"}}'
14 full
```

```
[root@ubuntu2204 ~]#awk -v test=100 'BEGIN{if(test>90){print "very good"} else if(test>60)
{print "good"} else{print "no pass"}}'
very good
```

```
[root@ubuntu2204 ~]#awk -v test=70 'BEGIN{if(test>90){print "very good"} else if(test>60)
{print "good"} else{print "no pass"}}'
good
```

```
[root@ubuntu2204 ~]#awk -v test=20 'BEGIN{if(test>90){print "very good"} else if(test>60)
{print "good"} else{print "no pass"}}'
no pass
```

条件判断 switch-case

awk 中的switch分支语句功能较弱，只能进行等值比较或正则匹配

各分支结尾需使用break来终止

语法:

```
switch(expression) {  
    case VALUE1 or /REGEXP1/:  
        statement1;  
        break;  
  
    case VALUE1 or /REGEXP1/:  
        statement1;  
        break;  
  
    .....;  
  
    default:  
        statement;  
        break;  
}
```

分支穿透:

```
switch(expression) {  
    case VALUE1 or /REGEXP1/:  
    case VALUE2 or /REGEXP2/:  
    case VALUE3 or /REGEXP3/:  
        statement1;  
        break;  
  
    case VALUE4 or /REGEXP4/:  
        statement2;  
        break;  
  
    .....  
  
    default:  
        statement;  
        break;  
}
```

范例:

```
[root@ubuntu2204 ~]#awk -F: '{switch($3){case 0:print "root",$3;break; case /^[1-9][0-9]{0,2}$|1000/:print "sys user",$3;break;default: print "common user", $3;break;}}'  
/etc/passwd | head -n20  
root 0  
sys user 1  
sys user 2  
sys user 3
```

```
sys user 4
sys user 5
sys user 6
sys user 7
sys user 8
sys user 9
sys user 10
sys user 13
sys user 33
sys user 34
sys user 38
sys user 39
sys user 41
common user 65534
sys user 100
sys user 101
```

循环 while

条件“真”，进入循环；条件“假”，退出循环

使用场景：对一行内的多个字段逐一类似处理时使用；对数组中的各元素逐一处理时使用

语法：

```
while (condition) {statement;...}
```

范例：

```
[root@ubuntu2204 ~]#awk -v i=1 -v sum=0 'BEGIN{while(i<=100){sum+=i;i++;print sum}}'
5050

[root@ubuntu2204 ~]#awk 'BEGIN{ total=0;i=1;while(i<=100){total+=i;i++;print total}}'
5050
```

循环 do-while

无论真假，至少执行一次循环

语法：

```
do {statement;...}while{condition}
```

范例：

```
[root@ubuntu2204 ~]#awk 'BEGIN{total=0;i=1;do{total+=i;i++;}while(i<=100);print total}'  
5050
```

循环 for

语法：

```
for(expr1;expr2;expr3) {statement;...}
```

特殊用法：能够遍历数组中的元素

```
for(var in array) {for-body}
```

范例：

```
[root@ubuntu2204 ~]#awk 'BEGIN{sum=0;for(i=1;i<=100;i++){sum+=i};print sum}'  
5050
```

范例：遍历数组

```
[root@ubuntu2204 ~]#awk 'BEGIN{ arr[0]=123;arr[1]=456;for(i in arr){print i ">=" arr[i]}}'  
0=>123  
1=>456
```

面试题：文件abc.txt只有一行数字，计算其总和


```
[root@ubuntu2204 ~]#cat abc.txt
1 2 3 4 5

[root@ubuntu2204 ~]#cat abc.txt |awk '{for(i=1;i<=NF;i++){sum+=i};print sum}'
15

[root@ubuntu2204 ~]#cat abc.txt|tr ' ' + |bc
15

[root@ubuntu2204 ~]#sum=0; for i in `cat abc.txt`; do let sum+=i; done;echo $sum
15
```

性能比较

```
[root@ubuntu2204 ~]#time (awk 'BEGIN{ total=0;for(i=0;i<=10000;i++){total+=i};print total;}')
50005000

real    0m0.003s
user    0m0.003s
sys     0m0.000s
[root@ubuntu2204 ~]#time (total=0;for i in {1..10000};do total=$((total+i));done;echo $total)
50005000

real    0m0.021s
user    0m0.016s
sys     0m0.004s
[root@ubuntu2204 ~]#time (for ((i=0;i<=10000;i++));do let total+=i;done;echo $total)
50005000

real    0m0.026s
user    0m0.026s
sys     0m0.000s
[root@ubuntu2204 ~]#time (seq -s"+" 10000|bc)
50005000

real    0m0.003s
user    0m0.004s
sys     0m0.000s
```

范例：取出字符串中的数字

```
# str=(str $i)简单解读
# 在 AWK 里，两个字符串相邻书写就表示拼接，不需要 + 或 .（像其他语言那样）
# 示例：
[root@ubuntu2204 ~]# awk 'BEGIN{s="abc"; s=s "def"; print s}'
abcdef

[root@ubuntu2204 ~]# echo 'dsFUs34tg*fs5a%8ar%$#@' | awk -F "" '{for(i=1;i<=NF;i++){if($i ~
/[0-9]/){str=(str $i)}}print str}'
3458
```

continue和break

continue 跳过本次循环

break中断整个循环

格式：

```
continue [n]
break [n]
```

范例：

```
[root@ubuntu2204 ~]#awk 'BEGIN{for(i=1;i<=100;i++){if(i==50)continue;sum+=i};print sum}'
5000

[root@ubuntu2204 ~]#awk 'BEGIN{for(i=1;i<=100;i++){if(i==50)break;sum+=i};print sum}'
1225
```

next

next 可以提前结束对本行处理而直接进入下一行处理（awk自身循环）

范例：

```
[root@ubuntu2204 ~]#awk -F: '{if($3%2!=0){next};; print $1,$3}' /etc/passwd
root 0
bin 2
sync 4
```

```
man 6
mail 8
uucp 10
backup 34
list 38
.....

[root@ubuntu2204 ~]#seq 10 |awk '{if($0%2!=0){next};{print $0}}'
2
4
6
8
10
```

数组

awk的数组为关联数组

格式

```
array_name{index-expression}
```

index-expression

- 利用数组，实现k/v功能
- 可使用任意字符串；字符串要使用双引号括起来
- 如果某数组元素事先不存在，在引用时，awk会自动创建此元素，并将其初始化为“空串”
- 若要判断数组中是否存在某元素，要使用"`index in array`"格式进行遍历

范例：

```
# 相当于创建一个字典{"mon"="Monday","tue":"Tuesday"}
[root@ubuntu2204 ~]#awk 'BEGIN{weekdays["mon"]="Monday";weekdays["tue"]="Tuesday";print
weekdays["mon"]}'
Monday
```

范例：判断数组索引是否存在

```
[root@ubuntu2204 ~]#awk 'BEGIN{array["i"]="x";array["j"]="y"; print "i" in array, "y" in array}'  
1 0
```

```
[root@ubuntu2204 ~]#awk 'BEGIN{array["i"]="x"; array["j"]="y"; if("i" in array) {print "存在"}else {print "不存在"}}'  
存在
```

```
[root@ubuntu2204 ~]#awk 'BEGIN{array["i"]="x"; array["j"]="y"; if("abc" in array) {print "存在"}else {print "不存在"}}'  
不存在
```

若要遍历数组中的每个元素，要使用 for 循环

```
for(var in array) {for-body}
```

范例：遍历数组

```
[root@ubuntu2204 ~]#awk 'BEGIN{weekdays["mon"]="Monday";weekdays["tue"]="Tuesday";for(i in weekdays){print i, weekdays[i]}}'  
tue Tuesday  
mon Monday
```

```
[root@ubuntu2204 ~]#awk  
'BEGIN{students[1]="user1";students[2]="user2";students[3]="user3";for(x in students){print x:"students[x]}}'  
1:user1  
2:user2  
3:user3
```

```
[root@ubuntu2204 ~]#awk 'BEGIN{  
> a["x"] = "welcome"  
> a["y"] = "to"  
> a["z"] = "Magedu"  
> for (i in a) {  
>     print i, a[i]  
> }  
> }'  
x welcome  
y to  
z Magedu
```

```
[root@ubuntu2204 ~]#awk -F: '{user[$1]=$3}END{for(i in user){print "username:"i,"uid:"user[i]}}' /etc/passwd  
username:gnats uid:41  
username:tcpdump uid:109  
username:sshd uid:106  
username:usbmux uid:112
```

```
username:syslog uid:107
username:systemd-resolve uid:102
username:systemd-network uid:101
username:bin uid:2
username:tss uid:110
username:pollinate uid:105
username:mail uid:8
.....
```

范例：

```
[root@ubuntu2204 ~]# awk '{ip[$1]++}END{for(i in ip){print i,ip[i]}}'
/www/wwwlogs/access.log |sort -nr -k2|head -n10
106.54.233.20 1080
129.226.147.239 542
112.124.11.25 531
47.92.65.140 477
47.236.251.77 447
47.236.251.7 350
47.122.64.1 301
79.124.56.186 260
123.56.28.124 209
80.66.83.46 150
```

范例：封掉查看访问日志中连接次数超过1000次的IP

```
[root@ubuntu2204 ~]# awk '{
    ip[$1]++
}
END {
    for(i in ip){
        if(ip[i] >= 1000){
            system("iptables -A INPUT -s " i " -j REJECT")
        }
    }
}' nginx.access.log-20250801
```

system() 是 AWK 的一个内置函数，用于在 AWK 脚本中执行外部系统命令。

示例：

```
[root@ubuntu2204 ~]#awk 'BEGIN {system("echo Hello from AWK")}'
```

Hello from AWK

范例：多维数组

```
# 下面的多维数组可以看成{1:{1:11,2:12,3:13},2:{1:21,2:22,3:23}}
```

```
[root@ubuntu2204 ~]#awk 'BEGIN{
> array[1][1]=11
> array[1][2]=12
> array[1][3]=13
> array[2][1]=21
> array[2][2]=22
> array[2][3]=23
> for (i in array)
>     for (j in array[i])
>         print array[i][j]
> }'
```

11
12
13
21
22
23

范例：

```
[root@ubuntu2204 ~]#cat score.txt
name sex score
alice f 100
bob m 90
ming m 95
hong f 90

[root@ubuntu2204 ~]#awk 'NR!=1{if($2=="m")
{m_sum+=$3;m_num++}else{f_sum+=$3;f_num++}}END{print "男生平均成绩="m_sum/m_num,"女生平均成绩
="f_sum/f_num}' score.txt
男生平均成绩=92.5 女生平均成绩=95

[root@ubuntu2204 ~]#awk 'NR!=1{score[$2]+=$3;num[$2]++}END{for(i in score){print
i,score[i]/num[i]}}' score.txt
m 92.5
f 95

[root@ubuntu2204 ~]#awk 'NR!=1{score[$2]+=$3;num[$2]++}END{for(i in score){if(i=="m"){print
"男生平均成绩=",score[i]/num[i]}else{print "女生平均成绩=",score[i]/num[i]}}}' score.txt
男生平均成绩= 92.5
女生平均成绩= 95
```

awk 函数

awk 的函数分为内置和自定义函数

官方文档

<https://www.gnu.org/software/gawk/manual/gawk.html#Functions>

常用内置函数

数值处理

<code>rand()</code>	# 返回0和1之间一个随机数
<code>srand()</code>	# 配合 <code>rand()</code> 函数,生成随机数的种子
<code>int()</code>	# 返回整数

范例:

```
[root@ubuntu2204 ~]#awk 'BEGIN{srand(); for(i=1;i<=5;i++)print int(rand()*100)}'
66
71
47
99
1
```

字符串处理

<code>str1=(str2,str3)</code>	# 连字符, 字符串拼接
<code>length([s])</code>	# 返回指定字符串的长度
<code>sub(r,s,[t])</code>	# 在 <code>t</code> 中串搜索 <code>r</code> 匹配的内容, 并将第一匹配内容替换为 <code>s</code>
<code>gsub(r,s,[t])</code>	# 在 <code>t</code> 中串搜索 <code>r</code> 匹配的内容, 并将匹配内容全部替换为 <code>s</code> , 即贪婪模式
<code>split(s,array,[r])</code>	# 以 <code>r</code> 为分隔符, 切割串 <code>s</code> , 并将结果保存至 <code>array</code> 数组中, 索引值为1,2,...

范例: 统计用户名的长度

```
[root@ubuntu2204 ~]#cut -d: -f1 /etc/passwd|awk '{print length()}'
.....
5
6
4
15
15
10
16
.....

[root@ubuntu2204 ~]#awk -F: '{print length($1)}' /etc/passwd
.....
5
6
```

```
4
15
15
10
16
.....
```

范例：

```
[root@ubuntu2204 ~]#netstat -tn|awk '/^tcp/{split($4,ip,":");count[ip[1]]++}END{for(i in count){print i,count[i]}}'
10.0.0.202 1
```

调用shell命令

```
system('cmd')
```

空格是awk中的字符串连接符，如果system中需要使用awk中的变量可以使用空格分隔，或者说除了 awk的变量外其他一律用""引用起来

范例：

```
[root@ubuntu2204 ~]#awk 'BEGIN{system("hostname")}'
ubuntu2204.wang.org

[root@ubuntu2204 ~]#awk 'BEGIN{score=100; system("echo your score is " score)}'
your score is 100
```

时间函数

systime()	# 当前时间到1970年1月1日的秒数，即时间戳
strftime()	# 指定时间格式

范例：

```
[root@ubuntu2204 ~]#awk 'BEGIN{print systime()}'
1754276394
[root@ubuntu2204 ~]#awk 'BEGIN{print systime()}'
1754276398
[root@ubuntu2204 ~]#awk 'BEGIN{print systime()}'
1754276399
```



```
[root@ubuntu2204 ~]#awk 'BEGIN{print systime();system("date +%s")}'
1754276451
1754276451

[root@ubuntu2204 ~]#awk 'BEGIN{print strftime("%Y-%m-%dT%H:%M", systime()-3600)}'
2025-08-04T10:02

[root@ubuntu2204 ~]#awk 'BEGIN{print strftime("%Y-%m-%d %H:%M:%S");system("date \"+%Y-%m-%d %H:%M:%S\"")}'
2025-08-04 11:03:15
2025-08-04 11:03:15
```

自定义函数

自定义函数格式

```
function name (param, param, ...) {
    statements
    return expression
}
```

范例:

```
[root@ubuntu2204 ~]# awk 'function test(){print "hello test func"}BEGIN{test()}'
hello test func

[root@ubuntu2204 ~]#awk -F: 'function test(uname,uid){print uname" id is "uid}{test($1,$3)}'
/etc/passwd
root id is 0
daemon id is 1
bin id is 2
sys id is 3
sync id is 4
games id is 5
man id is 6
lp id is 7
.....
```

awk脚本

将awk程序写成脚本，直接调用或执行

范例：

```
[root@ubuntu2204 awk]#cat passwd.awk
{if($3>=1000)print $1,$3}

[root@ubuntu2204 awk]#awk -F: -f passwd.awk /etc/passwd
nobody 65534
wang 1000
```

范例：

```
[root@ubuntu2204 awk]#cat test.awk
#!/bin/awk -f
{
    if($3 >= 1000)
        print $1,$3
}

[root@ubuntu2204 awk]#chmod +x test.awk
[root@ubuntu2204 awk]#./test.awk -F: /etc/passwd
nobody 65534
wang 1000
```

向 awk 脚本传递参数

格式：

```
awkfile var=value var2=value2... Inputfile
```

注意：

- 上面格式变量在 BEGIN 过程中不可用。直到首行输入完成以后，变量才可用
- 可以通过 `-v` 参数，让 awk 在执行BEGIN之前得到变量的值
- 命令行中每一个指定的变量都需要一个 `-v` 参数

关于 awk 变量赋值的时机解读

1. `-v x=200`

`-v` 选项 在 `AWK` 程序开始之前（包括 `BEGIN` 块）就设置了变量
所以 `-v` 定义的变量可以在 `BEGIN` 和所有后续代码块中使用。

2. `x=200`（放在文件名前）

当写到: `awk '...' x=200 file`, 这里的 `x=200` 是 命令行参数的一部分, 不是 `-v` 选项。
这类变量赋值 “发生在文件读取之后”, 即在 `BEGIN` 执行完后才生效。

因此:

- 在 `BEGIN` 中变量 `x` 还是默认空 (`0`)
- 在后续规则 `{}` 中 `x=200` 已经生效。

范例:

```
# BEGIN和后续{}内都生效
[root@ubuntu2204 awk]#awk -v x=100 'BEGIN{print x}{print x+100}' /etc/issue
100
200
200

# BEGIN内不生效
[root@ubuntu2204 awk]#awk 'BEGIN{print x}{print x+100}' x=200 /etc/issue

300
300
```

范例:

```
[root@ubuntu2204 awk]#cat test2.awk
#!/bin/awk -f
{
    if($3 >= min && $3 <= max)
        print $1,$3
}

[root@ubuntu2204 awk]#chmod +x test2.awk

[root@ubuntu2204 awk]#./test2.awk -F: min=100 max=200 /etc/passwd
apt 100
systemd-network 101
systemd-resolve 102
messagebus 103
systemd-timesync 104
pollinate 105
sshd 106
syslog 107
uidd 108
tcpdump 109
tss 110
```

```
landscape 111
usbmux 112
```

练习

1、文件host_list.log 如下格式，请提取“.magedu.com”前面的主机名部分并写入到回到该文件中

```
1 www.magedu.com
2 blog.magedu.com
3 study.magedu.com
4 linux.magedu.com
5 python.magedu.com
.....
999 study.magedu.com
```

参考答案：

```
[root@ubuntu2204 ~]# awk -F'[.]' '{print $2}' host_list.log >> host_list.log

[root@ubuntu2204 ~]# cat host_list.log
1 www.magedu.com
2 blog.magedu.com
3 study.magedu.com
4 linux.magedu.com
5 python.magedu.com
.....
999 study.magedu.com
www
blog
study
linux python
```

2、统计/etc/fstab文件中每个文件系统类型出现的次数

参考答案

```
[root@rocky86 ~]# awk -F' +' '/^UUID/{fs[$3]++}END{for(i in fs){print i,fs[i]}}' /etc/fstab
swap 1
ext4 3
```

3、统计/etc/fstab文件中每个单词出现的次数

参考答案

```
[root@ubuntu2204 awk]#awk -F"[^[:alpha:]]" '{for(i=1;i<=NF;i++)word[$i]++}END{for (a in word)if(a != "") print a,word[a]}' /etc/fstab
```

4、提取出字符串Yd\$C@M05MB%9&Bdh7dq+YVixp3vpw中的所有数字

参考答案

```
[root@ubuntu2204 awk]#echo 'Yd$C@M05MB%9&Bdh7dq+YVixp3vpw' | awk '{gsub(/[0-9]/,"");print $0}'  
05973
```

```
[root@ubuntu2204 awk]#echo 'Yd$C@M05MB%9&Bdh7dq+YVixp3vpw' | awk -F "" '{for(i=1;i<=NF;i++){if ($i ~ /[[:digit:]]/){str=$i; str1=(str1 str)}};print str1}'  
05973
```

```
[root@ubuntu2204 awk]#echo 'Yd$C@M05MB%9&Bdh7dq+YVixp3vpw' | awk -F '[^0-9]' '{for(i=1;i<=NF;i++){printf "%s", $i }}'  
05973
```

5、文件 random.txt 记录共5000个随机的整数，存储的格式100,50,35,89...请取出其中最大和最小的整数

参考答案

```
[root@ubuntu2204 awk]# str="";for((i=1;i<=5000;i++));do if [ $i -ne 5000 ];then  
str+="$RANDOM,";else str+=$RANDOM;fi;done;echo $str > random.txtcat
```

```
[root@ubuntu2204 awk]#awk -F, '{max=$1;min=$1;for(i=1;i<=NF;i++){if($i>max)  
{max=$i}else{if($i<min){min=$i}}}}END{print "最大值: "max,"最小值: "min}' random.txt
```

最大值: 32767 最小值: 5w

6、解决Dos攻击生产案例：监控当某个IP并发连接数超过100时，即调用防火墙命令封掉对应的IP，监控频率每隔5分钟。防火墙命令为：`iptables -A INPUT -s IP -j REJECT`

参看答案

```
[root@ubuntu2204 awk]#ss -nt | awk -F " +|:" 'NR!=1{ip[$(NF-2)]++}END{for(i in ip)  
{if(ip[i]>100) {system("iptables -A INPUT -s "i" -j REJECT")}}}'
```

7、将以下文件内容中FQDN取出并根据其进行计数从高到低排序

```
http://mail.magedu.com/index.html
http://www.magedu.com/test.html
http://study.magedu.com/index.html
http://blog.magedu.com/index.html
http://www.magedu.com/images/logo.jpg
http://blog.magedu.com/20080102.html
http://www.magedu.com/images/magedu.jpg
```

参考答案

```
[root@ubuntu2204 awk]#awk -F"/" '{url[$3]++}END{for(i in url){print url[i],i}}' url.log
|sort -nr
3 www.magedu.com
2 blog.magedu.com
1 study.magedu.com
1 mail.magedu.com
```

8、将以下文本文件awktest.txt中以inode列为标记，对inode列相同的counts列进行累加，并且统计出 同一inode中，beginnumber列中的最小值和endnumber列中的最大值

```
[root@ubuntu2204 awk]#cat awktest.txt
inode|beginnumber|endnumber|counts|
106|3363120000|3363129999|10000|
106|3368560000|3368579999|20000|
310|3337000000|3337000100|101|
310|3342950000|3342959999|10000|
310|3362120960|3362120961|2|
311|3313460102|3313469999|9898|
311|3313470000|3313499999|30000|
311|3362120962|3362120963|2|
```

参考答案

```
[root@ubuntu2204 awk]#awk -F '|' '!/^inode/{sum[$1]+=$4;if(!begin[$1])begin[$1]=$2}else
if(begin[$1]>$2)begin[$1]=$2;if(!end[$1])end[$1]=$3}else if(end[$1] <$3)end[$1]=$3}END{for(i
in sum)print i|"begin[i]"|end[i]"|sum[i]}' awktest.txt
106|3363120000|3368579999|30000
310|3337000000|3362120961|10103
311|3313460102|3362120963|39900

[root@ubuntu2204 awk]#cat test3.awk
#!/bin/awk -f

!/^inode/{
    sum[$1]+=$4
    if(!begin[$1])
```

```
        begin[$1]=$2
    else if(begin[$1]>$2)
        begin[$1]=$2
    if(!end[$1])
        end[$1]=$3
    else if(end[$1]<$3)
        end[$1]=$3
}

END{
    for(i in sum)
        print i|"begin[i]"|end[i]"|sum[i]
}

[root@ubuntu2204 awk]#chmod +x test3.awk

[root@ubuntu2204 awk]#./test3.awk -F "|" awktest.txt
106|3363120000|3368579999|30000
310|3337000000|3362120961|10103
311|3313460102|3362120963|39900
```